Homework 9

15.2170.)

a.

We'll prove that if A, B are PSD and $\alpha \in \mathbb{R}$, $\alpha \geq 0$, then αA and $A + B$ are PSD as well. To prove this, we want to show that for any vector x,

$$x^\top(\alpha A)x \geq 0$$
$$x^\top(A + B)x \geq 0$$

Fix arbitrary vector x. Since A, B are PSD, we have

$$x^\top Ax \geq 0$$
$$x^\top Bx \geq 0$$

For the first statement, we have $\alpha \geq 0$ and $x^\top Ax \geq 0$ imply

$$x^\top(\alpha A)x = \alpha(x^\top Ax) \geq 0$$

Thus the first statement holds. For the second statement, we have $x^\top Ax \geq 0$ and $x^\top Bx \geq 0$ imply

$$x^\top(A + B)x = x^\top Ax + x^\top Bx \geq 0$$

Therefore the second statement holds. Since x was arbitrary, this holds for any vector. Hence, if A, B are PSD and $\alpha \in \mathbb{R}$, $\alpha \geq 0$, then αA and $A + B$ are PSD as well.

b.

We'll prove that any symmetric submatrix of a PSD matrix is PSD. Assume for contradiction this isn't true. Then there exists matrix A which is PSD and symmetric submatrix $\tilde{A}$ of A and vector $\tilde{x}$ s.t.

$$\tilde{x}^\top \tilde{A}\tilde{x} < 0$$

We constructed $\tilde{A}$ from A by choosing a subset of $\{1, ..., n\}$ and throwing away all the other columns and rows. Construct x from $\tilde{x}$ by padding $\tilde{x}$ with corresponding zeroes to make it a sort of inverse transformation. In other words, if we construct $\tilde{A}$ from A by choosing $\{1, 3, 4\} \subseteq \{1, 2, 3, 4, 5\}$ then we can write $\tilde{x}$ as $\tilde{x} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}^\top$, so we'd construct

$$x = \begin{bmatrix} x_1 \\ 0 \\ x_2 \\ x_3 \\ 0 \end{bmatrix}$$

From here, we have

$$x^\top Ax = \tilde{x}^\top \tilde{A}\tilde{x} < 0$$

This contradicts A being PSD. Hence, by contradiction, any symmetric submatrix of a PSD matrix is PSD.

c.

We'll prove that if A is PSD then $A_{ii} \geq 0$ for all i. Since A is PSD we have for any vector x,

$$x^\top Ax \geq 0$$

Note that for arbitrary e_i, we have

$$e_i^\top Ae_i = A_{ii}$$

Since e_i is a vector, we can use the first inequality to find

$$A_{ii} = e_i^\top Ae_i \geq 0$$

As e_i was arbitrary, this holds for all i. Hence, if A is PSD then $A_{ii} \geq 0$ for all i.

d.

We'll prove that if A is symmetric PSD then $|A_{ij}| \leq \sqrt{A_{ii}A_{jj}}$ for all i, j and if $A_{ii} = 0$ then the entire ith row and column of A are zero. Since A is PSD we have for any vector x,

$$x^\top Ax \geq 0$$

Consider $x = e_i + te_j$ for arbitrary $t \in \mathbb{R}$ and arbitrary i, j. Note that since A is symmetric, we have $A_{ij} = A_{ji}$. Then we have

$$(e_i + te_j)^\top A(e_i + te_j) = e_i^\top Ae_i + te_i^\top Ate_j + te_j^\top Ae_i + t^2 e_j^\top Ae_j = A_{jj}t^2 + 2A_{ij}t + A_{ii} \geq 0$$

As t was arbitrary, this inequality holds for all $t \in \mathbb{R}$. Consider the strict equality $A_{jj}t^2 + 2A_{ij}t + A_{ii} = 0$. If $A_{jj} \neq 0$, we can use the quadratic formula to solve for t

$$t = \frac{-2A_{ij} \pm \sqrt{(2A_{ij})^2 - 4A_{ii}A_{jj}}}{2A_{jj}}$$

Notice that since $A_{jj}t^2 + 2A_{ij}t + A_{ii}$ is a quadratic polynomial and is ≥ 0 for all t, we have $A_{jj}t^2 + 2A_{ij}t + A_{ii} = 0$ has at most one solution. If $(2A_{ij})^2 - 4A_{ii}A_{jj} > 0$, then we would have two distinct solutions, which is a contradiction. Thus, we must have

$$(2A_{ij})^2 - 4A_{ii}A_{jj} \leq 0 \Leftrightarrow 4A_{ij}^2 \leq 4A_{ii}A_{jj} \Leftrightarrow |A_{ij}| \leq \sqrt{A_{ii}A_{jj}}$$

Now consider $A_{jj} = 0$. Then we have $2A_{ij}t + A_{ii} \geq 0$ for all t. A non-constant linear function cannot be ≥ 0 for all t, which means we must have $2A_{ij} = A_{ij} = 0$. Thus,

$$0 = |A_{ij}| \leq \sqrt{A_{ii}A_{jj}} = 0$$

As i, j were arbitrary, this holds for all i, j. Therefore, if A is symmetric PSD then $|A_{ij}| \leq \sqrt{A_{ii}A_{jj}}$ for all i, j. From this conclusion, we can see that if $A_{ii} = 0$ then the entire ith row and column of A are zero. To fully show this, assume for contradiction we have a symmetric PSD matrix A where $A_{ii} = 0$ but there's a nonzero entry in the ith row or the ith column of A. Since A is symmetric, that means if there's a nonzero entry in the ith column of A then there's a nonzero entry in the ith row of A. In other words, there must exist nonzero A_{ij} s.t. $i \neq j$. We then get

$$0 \neq |A_{ij}| > \sqrt{A_{ii}A_{jj}} = 0$$

This is a contradiction. Hence, if A is symmetric PSD where $A_{ii} = 0$, then the entire ith row and column of A are zero.

15.2320.)

Given $A \in \mathbb{R}^{m \times n}$ s.t. $m \geq n$ and A is full rank, we want to find $F \in \mathbb{R}^{n \times m}$ that minimizes

$$\|I - FA\|$$

s.t. $\|F\| \leq \alpha$. Let $A = U\Sigma V^\top$ be the SVD of A where $\Sigma = \text{diag}(\sigma_1, \ldots, \sigma_n)$, $U = [u_1 \ \cdots \ u_n]$, $V = [v_1 \ \cdots \ v_n]$.
We've proven previously that for arbitrary matrix A and orthogonal matrix K,

$$\|A\| = ||KA\| = \|AK\|$$

For any candidate F, define

$$G = V^\top FU$$

Then

$$FA = F(U\Sigma V^\top) = (VGU^\top)(U\Sigma V^\top) = V(G\Sigma)V^\top$$

Using the orthogonal property from earlier,

$$\|I - FA\| = \|VV^\top - V(G\Sigma)V^\top\| = \|V(I - G\Sigma)V^\top\| = \|I - G\Sigma\|$$

$$\|F\| = \|VGU^\top\| = \|G\|$$

Thus the original problem is equivalent to finding $G \in \mathbb{R}^{n \times n}$ that minimizes

$$\|I - G\Sigma\|$$

s.t. $\|G\| \leq \alpha$. Note that off diagonal terms for a matrix can only increase the operator norm. Since Σ is diagonal, $I - G\Sigma$ has the smallest operator norm when $G\Sigma$ is diagonal as well, because this is a sufficient condition for $I - G\Sigma$ to be diagonal. This means WLOG we can assume that G is diagonal. So we can write $G = \text{diag}(g_1, \ldots, g_n)$. For a diagonal matrix G we have $\|G\| = \max\{|g_1|, ..., |g_n|\}$, so we rewrite our constraint to be $|g_i| \leq \alpha$ for $i = 1, ..., n$ and rewrite the objective to be

$$\min_G \|I - G\Sigma\| = \min_{|g_i| \leq \alpha} \|I - G\Sigma\| = \min_{|g_i| \leq \alpha} \max_i |1 - g_i\sigma_i|$$

So for each σ_i, we want to find $g_i \in [-\alpha, \alpha]$ that minimizes $|1 - g_i\sigma_i|$. If $\frac{1}{\sigma_i} \in [-\alpha, \alpha]$, then $g_i = \frac{1}{\sigma_i}$ is optimal. If $\frac{1}{\sigma_i} \notin [-\alpha, \alpha]$, then we know $\frac{1}{\sigma_i} > \alpha$ since $\sigma_i > 0$ by SVD properties and the best we can do is choose the largest allowed value $g_i = \alpha$ which gives the minimal achievable value of $|1 - g_i\sigma_i| = 1 - \alpha\sigma_i$. Therefore

$$g_i^\star = \min\left(\frac{1}{\sigma_i},\ \alpha\right)$$

Hence an optimal solution is

$$F^\star = V \,\text{diag}\big(g_1^\star, \ldots, g_n^\star\big)\, U^\top$$

where $g_i^\star$ is defined as above. The optimal objective value is

$$\|I - F^\star A\| = \max_i |1 - g_i^\star\sigma_i| = \max(0,\ 1 - \alpha\sigma_{\min})$$

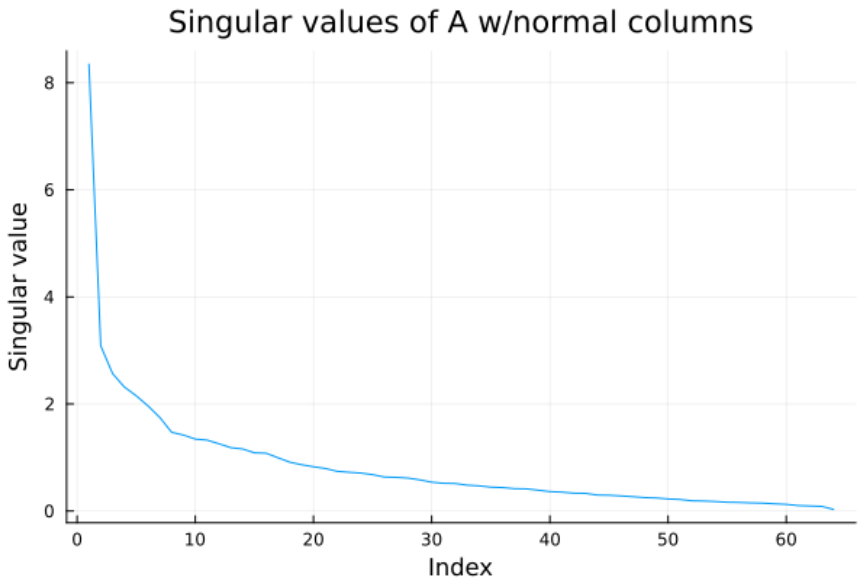
where $\sigma_{\min} = \min_i \sigma_i$. This means if $\alpha \geq 1/\sigma_{\min}$, then $\|I - F^\star A\| = 0$.

16.2710.)

a.

This question was done in Julia, the code and plot can be seen below.

```julia
using LinearAlgebra, Plots

# Question 3

include("readclassjson.jl")
data = readclassjson("term_by_doc.json")
A = data["A"]
term = data["term"]
document = data["document"]
m = data["m"]
n = data["n"]

# Part a
norms = [norm(col) for col in eachcol(A)]
A_tilde = A ./ reshape(norms, 1, :)
U, S, V = svd(A_tilde)

# Plot singular values
plot(S, xlabel="Index",
        ylabel="Singular value",
        title="Singular values of A w/normal columns",
        label="")
savefig("hw9_q3a.png")

```



b. + c.

These questions were done in Julia, the code and output can be seen below. As can be seen in the output, the top 5 search results for each of the matrices are

Rank	$\mathbf{A}_{full}$	$\mathbf{A}_{32}$	$\mathbf{A}_{16}$	$\mathbf{A}_{8}$	$\mathbf{A}_{4}$
1	106	106	106	115	115
2	105	105	107	106	105
3	107	107	105	120	107
4	115	115	115	107	66
5	111	111	111	111	63

For the full-rank and 32-rank approximations, the top five documents match exactly, and the 16-rank result differs only by a small permutation of the second and third entries. However, the 8-rank and 4-rank approximations change both the ordering and which documents appear in the top 5, with the 4-rank approximation even failing to recover the top document from the full-rank matrix. This behavior shows the effect of reducing the rank: a moderate reduction (such as with ranks 32 and 16) preserves the dominant structure of the matrix, while very low ranks (such as with 8 and 4) begin to lose important information.

That said, low-rank approximations are valuable because they perform latent semantic indexing. Even though extremely low ranks highly change the top results, moderate rank reduction can improve search results by denoising the matrix and capturing latent associations between terms and documents. In this example, a query for "students" with a moderate rank reduction might retrieve documents that do not explicitly contain the word "students" but frequently mention related terms such as "school" or "course", because these relationships are encoded in the leading singular directions. Thus, rank 32 and 16 preserve both accuracy and the semantic structure needed for latent semantic indexing, while ranks 8 and 4 might be too small to capture the relevant meaning.

```
25  # Part b
26  q = zeros(m)
27  q[53] = 1
28  c = A_tilde' * q
29  p = sortperm(c, rev=true)
30  println("Top 5 search results indices, full matrix:  ", p[1:5])
```

```
Top 5 search results indices, full matrix:  [106, 105, 107, 115, 111]
```

```
32  # Part c
33  ranks = [32, 16, 8, 4]
34  for r in ranks
35      temp = svd(A_tilde)
36      A_hat = temp.U[:, 1:r] * Diagonal(temp.S[1:r]) * temp.Vt[1:r, :]
37      c_hat = A_hat' * q
38      p_hat = sortperm(c_hat, rev=true)
39      println("Top 5 search results indices, low rank matrix s.t. r = $r:  ", p_hat[1:5])
40  end
```

```
Top 5 search results indices, low rank matrix s.t. r = 32:  [106, 105, 107, 115, 111]
Top 5 search results indices, low rank matrix s.t. r = 16:  [106, 107, 105, 115, 111]
Top 5 search results indices, low rank matrix s.t. r = 8:  [115, 106, 120, 107, 111]
Top 5 search results indices, low rank matrix s.t. r = 4:  [115, 105, 107, 66, 63]
```

d.

There are advantages in using low-rank approximations over the full matrix. The main benefit is that a low-rank approximation significantly reduces the number of matrix operations required for each search, making the computation faster and more efficient. Since a very large number of searches will be performed before the term-by-document matrix is updated, this reduction in per-query cost can lead to substantial savings. Lower rank approximations require fewer operations, so the computation becomes cheaper as the rank decreases.

A second benefit is that low-rank approximations perform latent semantic indexing. By keeping only the most impactful singular vectors, the approximation removes noise and reveals underlying semantic structure in the data. This allows the

system to retrieve documents that may not contain the query word but contain related terms. For instance, a query for "students" might retrieve a document discussing "university" even if the word "students" never appears.

Hence, low-rank approximations are both more efficient and often more semantically informative than using the full-rank matrix.

17.1340.)

a.

For the linear Gaussian model, the estimator gain is

$$L_{\text{mmse}} = \Sigma_x A^\top \left(A\Sigma_x A^\top + \Sigma_w\right)^{-1}$$

where

$$A = \begin{bmatrix} \frac{q_1^\top}{\|q_1\|} \\ \frac{q_2^\top}{\|q_2\|} \\ \frac{q_3^\top}{\|q_3\|} \end{bmatrix}$$

$$q_1 = \begin{bmatrix} 500 \\ 0 \end{bmatrix}, \quad q_2 = \begin{bmatrix} 0 \\ 500 \end{bmatrix}, \quad q_3 = \begin{bmatrix} 500 \\ -200 \end{bmatrix}$$

$$\Sigma_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\Sigma_w = \begin{bmatrix} 0.1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

We computed L_{mmse} in Julia, the code and output are below. As can be seen in the output,

$$L_{\text{mmse}} = \begin{bmatrix} 0.847 & 0.014 & 0.074 \\ 0.137 & 0.470 & -0.162 \end{bmatrix}$$

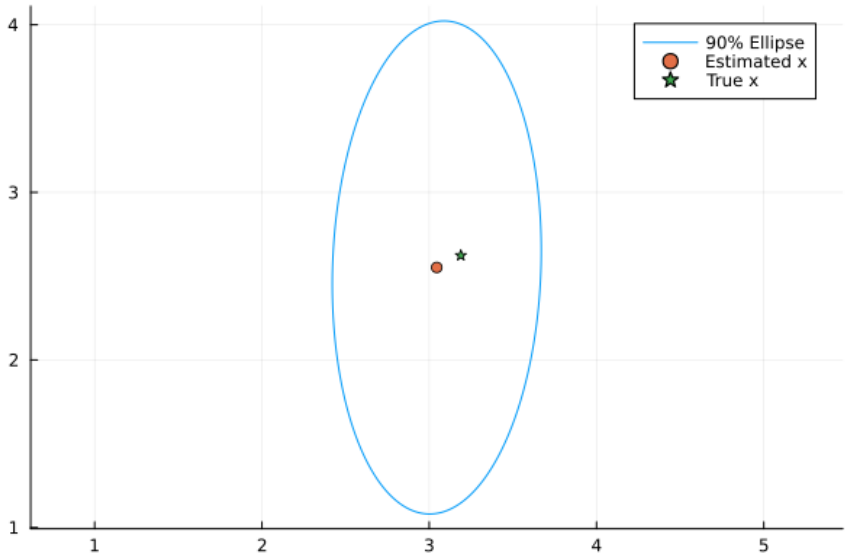
```julia
using LinearAlgebra, Distributions, Plots

#Question 4.)

#Part a
q1 = [500.0, 0.0]
q2 = [0.0, 500.0]
q3 = [500.0, -200.0]

A = [(q1' / norm(q1));
     (q2' / norm(q2));
     (q3' / norm(q3))]

sigma_x = I(2)
sigma_w = diagm([0.1, 1.0, 1.0])

Lmmse = sigma_x * A' * inv(A * sigma_x * A' + sigma_w)
println("L_mmse = ", Lmmse)
```

```
L_mmse = [0.8469945355191256 0.01366120218579235 0.07356782523407791; 0.1366120218579235 0.46994535519125685 -0.16184921551497145]
```

b.

This question was done in Julia, the code and plot can be seen below.

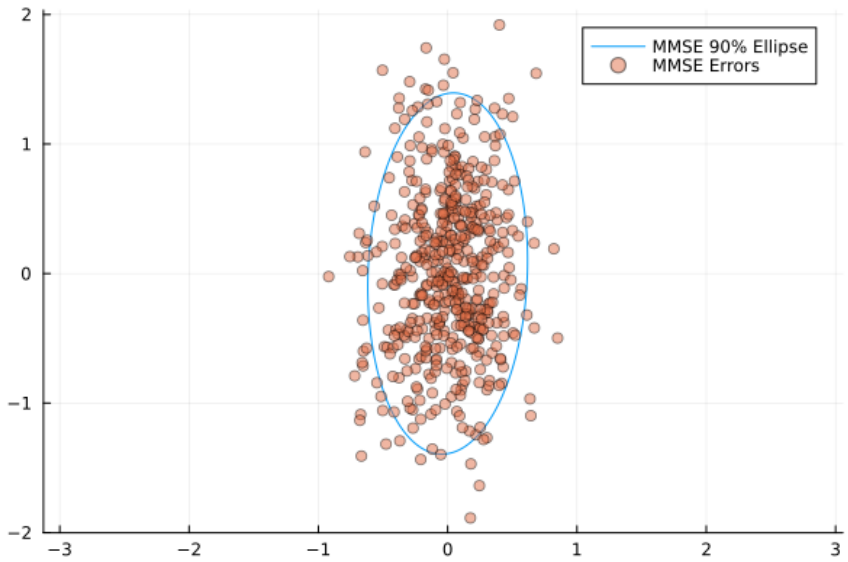
```julia
#Part b
mu_x = [2.0, 3.0]
mu_w = zeros(3)

x = rand(MvNormal(mu_x, sigma_x))
w = rand(MvNormal(mu_w, sigma_w))
y = A * x + w

mu_mmse = mu_x + Lmmse * (y - A * mu_x)
sigma_mmse = sigma_x - Lmmse * A * sigma_x

theta = range(0, 2*pi, length=400)
c = sqrt(4.605)           # chi2 for 90%
eigvals, eigvecs = eigen(sigma_mmse)

ellipse = [mu_mmse .+ c * eigvecs * diagm(sqrt.(eigvals)) * [cos(t), sin(t)] for t in theta]
xs = [p[1] for p in ellipse]
ys = [p[2] for p in ellipse]

plot(xs, ys, aspect_ratio=:equal, label="90% Ellipse")
scatter!([mu_mmse[1]], [mu_mmse[2]], label="Estimated x")
scatter!([x[1]], [x[2]], label="True x", marker=:star)
savefig("hw9_q4b.png")
```



c.

This question was done in Julia, the code and plot can be seen below.

```julia
#Part c
N = 500
x_samples = rand(MvNormal(mu_x, sigma_x), N)
w_samples = rand(MvNormal(mu_w, sigma_w), N)
A_pinv = pinv(A)
mmse_errors = zeros(2, N)
ls_errors = zeros(2, N)

for k in 1:N
    x = x_samples[:, k]
    w = w_samples[:, k]
    y = A*x + w

    mmse_errors[:, k] = x - (mu_x + Lmmse * (y - A * mu_x))
    ls_errors[:, k] = x - A_pinv*y
end

sigma_mmse_err = cov(eachcol(mmse_errors))
eigvals, eigvecs = eigen(sigma_mmse_err)
theta = range(0, 2π, length=400)
c = sqrt(4.605)   # chi2 for 90%

ellipse_mmse = [c * eigvecs * diagm(sqrt.(eigvals)) * [cos(t), sin(t)] for t in theta]
xs_mmse = [p[1] for p in ellipse_mmse]
ys_mmse = [p[2] for p in ellipse_mmse]

plot(xs_mmse, ys_mmse, aspect_ratio=:equal, label="MMSE 90% Ellipse")
scatter!(mmse_errors[1, :], mmse_errors[2, :], alpha=0.5, label="MMSE Errors")
savefig("hw9_q4c.png")
```



d.

Consider for $y = Ax + w$,

$$\phi_{\text{ls}}(y) = A^\dagger y = A^\dagger(Ax + w) = A^\dagger Ax + A^\dagger w = x + A^\dagger w$$

where $A^\dagger = (A^\top A)^{-1}A^\top$. Then we have

$$\mathrm{E}\big[\|\phi_{\text{ls}}(y) - x\|^2\big] = \mathrm{E}\big[(A^\dagger w)^\top (A^\dagger w)\big] = \mathrm{E}\big[w^\top (A^\dagger)^\top A^\dagger w\big] = \mathrm{E}\big[\mathrm{trace}\big((A^\dagger)^\top A^\dagger w w^\top\big)\big] \quad (\text{since } a^\top M a = \mathrm{tr}(Maa^\top))$$

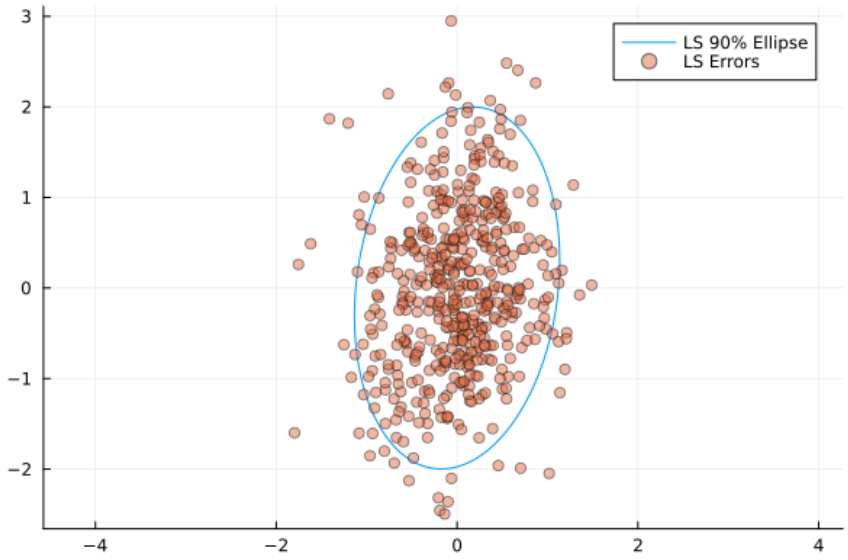
$$= \mathrm{trace}\big((A^\dagger)^\top A^\dagger \,\mathrm{E}[ww^\top]\big) = \mathrm{trace}\big((A^\dagger)^\top A^\dagger \,\Sigma_w\big) = \mathrm{trace}\big(A^\dagger \,\Sigma_w (A^\dagger)^\top\big) \quad (\text{cyclic property of trace})$$

$$= \mathrm{trace}\Big((A^\top A)^{-1} A^\top \Sigma_w A (A^\top A)^{-1}\Big)$$

e.

This question was done in Julia, the code and plot can be seen below.

```julia
#Part e
sigma_ls_err = cov(eachcol(ls_errors))
eigvals_ls, eigvecs_ls = eigen(sigma_ls_err)
theta = range(0, 2π, length=400)
c = sqrt(4.605)   # chi2 for 90%

ellipse_ls = [c * eigvecs_ls * diagm(sqrt.(eigvals_ls)) * [cos(t), sin(t)] for t in theta]
xs_ls = [p[1] for p in ellipse_ls]
ys_ls = [p[2] for p in ellipse_ls]

plot(xs_ls, ys_ls, aspect_ratio=:equal, label="LS 90% Ellipse")
scatter!(ls_errors[1, :], ls_errors[2, :], alpha=0.5, label="LS Errors")
savefig("hw9_q4e.png")
```



f.

The 90% confidence ellipsoid for the MMSE estimator in part c is always contained in the 90% confidence ellipsoid for the least-squares estimator in part e. Let $\phi_{\text{mmse}}(y) = \mathrm{E}[x \mid y]$ be the MMSE estimator and let $\phi(y)$ be any other estimator. Define the errors

$$z = x - \phi_{\text{mmse}}(y)$$

$$d(y) = \phi_{\text{mmse}}(y) - \phi(y)$$

Then we have

$$x - \phi(y) = z + d(y)$$

Using the orthogonality principle in question 17.1365, $\mathrm{E}[z\, d(y)^\top] = 0$. Therefore,

$$\Sigma_\phi = \mathrm{E}\big[(x - \phi(y))(x - \phi(y))^\top\big] = \mathrm{E}\big[(z + d(y))(z + d(y))^\top\big] = \mathrm{E}[zz^\top] + \mathrm{E}[dd^\top] = \Sigma_{\text{mmse}} + \mathrm{E}[dd^\top]$$

where Σ_ϕ is the arbitrary estimator error covariance and $\Sigma_{\text{mmse}} = \mathbb{E}[zz^\top]$ is the MMSE error covariance. Since $\mathrm{E}[dd^\top] \geq 0$, we get

$$\Sigma_\phi \geq \Sigma_{\text{mmse}}$$

For every other estimator ϕ. This includes the least squares estimator. Now let $E_1 = \{z :\ z^\top \Sigma_{\text{mmse}}^{-1} z \leq \chi_2^2(0.9)\}$ and $E_2 = \{z :\ z^\top \Sigma_{\text{ls}}^{-1} z \leq \chi_2^2(0.9)\}$ be the two 90% ellipsoids. Since we know $\Sigma_{\text{ls}} \geq \Sigma_{\text{mmse}} > 0$, we then have $\Sigma_{\text{mmse}}^{-1} \geq \Sigma_{\text{ls}}^{-1}$. Using the properties of ellipsoids, we know this implies $E_1 \subseteq E_2$. Thus the MMSE 90% ellipsoid is always contained in the LS 90% ellipsoid.

17.1360.)

Let x, w and y be as defined in the problem. We want to find the MMSE estimate $\mathrm{E}[x \mid y]$. Since $y = Ax + w$ and x, w are both Gaussian, we know that y is Gaussian as well. For a jointly Gaussian vector $\begin{bmatrix} x \\ y \end{bmatrix}$,

$$\mathrm{E}[x \mid y] = \mu_x + \Sigma_{xy}\,\Sigma_y^{-1}\,(y - \mu_y)$$

This means we just need to find μ_y, Σ_{xy} and Σ_y. For μ_y,

$$\mu_y = \mathrm{E}[y] = \mathrm{E}[Ax + w] = A\mathrm{E}[x] + \mathrm{E}[w] = A\mu_x + \mu_w$$

For Σ_{xy},

$$\Sigma_{xy} = \mathrm{E}\big[(x - \mu_x)(y - \mu_y)^\top\big] = \mathrm{E}\Big[(x - \mu_x)\big(A(x - \mu_x) + (w - \mu_w)\big)^\top\Big]$$
$$= \mathrm{E}\big[(x - \mu_x)\big(x - \mu_x\big)^\top\big]\,A^\top + \mathrm{E}\big[(x - \mu_x)(w - \mu_w)^\top\big] = \Sigma_x A^\top + \Sigma_{xw}$$

For Σ_y,

$$\Sigma_y = \mathrm{E}\big[(y - \mu_y)(y - \mu_y)^\top\big] = \mathrm{E}\Big[\big(A(x - \mu_x) + (w - \mu_w)\big)\big(A(x - \mu_x) + (w - \mu_w)\big)^\top\Big]$$
$$= A\mathrm{E}\big[(x - \mu_x)\big(x - \mu_x\big)^\top\big]\,A^\top + A\mathrm{E}\big[(x - \mu_x)\big(w - \mu_w\big)^\top\big] + \mathrm{E}\big[(w - \mu_w)\big(x - \mu_x\big)^\top\big]\,A^\top + \mathrm{E}\big[(w - \mu_w)\big(w - \mu_w\big)^\top\big]$$
$$= A\Sigma_x A^T + A\Sigma_{xw} + \Sigma_{wx}A^T + \Sigma_w$$

Hence, we have

$$\mathrm{E}[x \mid y] = \mu_x + \big(\Sigma_x A^\top + \Sigma_{xw}\big)\big(A\Sigma_x A^\top + A\Sigma_{xw} + \Sigma_{wx}A^\top + \Sigma_w\big)^{-1}\big(y - (A\mu_x + \mu_w)\big)$$

17.1365.)

a.

Let $z = x - \phi(y)$ and $\phi(y) = \mathrm{E}[x \mid y]$. Then we have

$$\mathrm{E}[zy^\top] = \mathrm{E}[(x - \phi(y))y^\top] = \mathrm{E}[xy^\top] - \mathrm{E}[\phi(y)y^\top] = \mathrm{E}[xy^\top] - \mathrm{E}[\mathrm{E}[x \mid y]y^\top]$$

We can then further break down $\mathrm{E}[\mathrm{E}[x \mid y]y^\top]$

$$\mathrm{E}[x \mid y] = \int xp^{|y}(x, y)dx = \frac{1}{p^y(y)} \int xp(x, y)dx$$

$$\mathrm{E}[\mathrm{E}[x \mid y]y^\top] = \int \mathrm{E}[x \mid y]y^\top p^y(y)dy = \int \int xy^\top p(x, y)dxdy = E[xy^\top]$$

Hence,

$$\mathrm{E}[zy^\top] = \mathrm{E}[xy^\top] - \mathrm{E}[\mathrm{E}[x \mid y]y^\top] = \mathrm{E}[xy^\top] - \mathrm{E}[xy^\top] = 0$$

b.

I would expect the estimation error z to be uncorrelated with the measurement y because the estimator $\phi(y)$ uses all the information that can be provided by y to help predict x. In other words, if z were correlated with y, then some component of the error could still be predicted from y. In that case, we could modify $\phi(y)$ by adding a function of y to account for the correlation and subsequently reduce the mean square error. However, the MMSE estimator is optimal, so no such improvement is possible. This implies the estimation error is uncorrelated with y, supporting our result in part a.